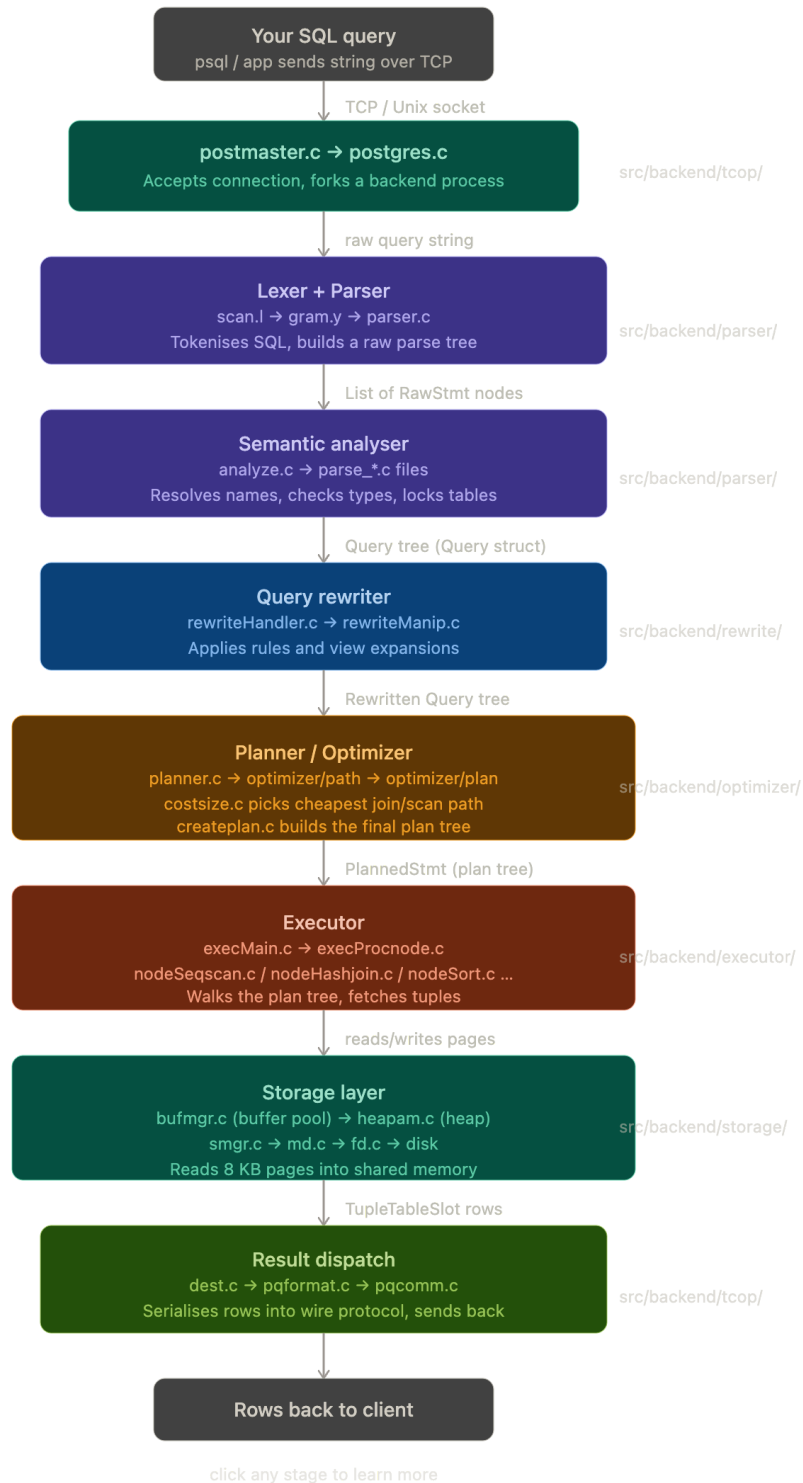


QUERY EXECUTION FLOW :



Stage 1 — Connection (`src/backend/tcop/`)

Files	<code>postmaster.c</code> , <code>postgres.c</code>
Input	Raw TCP/Unix socket connection from client
What happens	<code>postmaster</code> accepts the connection and forks a new backend process. <code>postgres.c</code> → <code>PostgresMain()</code> is the entry point for that backend
Output	A live backend process waiting for a query string

Stage 2 — Lexer + Parser (`src/backend/parser/`)

Files	<code>scan.l</code> → <code>gram.y</code> → <code>parser.c</code> → <code>parsetree.h</code>
Input	Raw SQL string e.g. <code>"SELECT * FROM customers"</code>
What happens	<code>scan.l</code> (flex) tokenises the string into keywords/identifiers. <code>gram.y</code> (bison) matches tokens against grammar rules and builds a raw parse tree
Output	<code>List *</code> of <code>RawStmt</code> nodes (raw, unresolved — table names are just strings at this point)

Stage 3 — Semantic Analyser (`src/backend/parser/`)

Files	<code>analyze.c</code> , <code>parse_relation.c</code> , <code>parse_expr.c</code> , <code>parse_type.c</code>
Input	<code>List *</code> of <code>RawStmt</code> nodes
What happens	Looks up table names in the catalog, resolves column names, checks data types, acquires table locks
Output	<code>Query *</code> struct — a fully resolved query tree with OIDs instead of names

Stage 4 — Rewriter (`src/backend/rewrite/`)

Files	<code>rewriteHandler.c</code> , <code>rewriteManip.c</code> , <code>rewriteDefine.c</code>

Input	<code>Query *</code> struct
What happens	Expands views into their underlying queries, applies <code>CREATE RULE</code> rules
Output	A new (possibly expanded) <code>Query *</code> — could be multiple queries if rules fire

Stage 5 — Planner/Optimizer (`src/backend/optimizer/`)

Files	<code>planner.c</code> , <code>optimizer/path/costsize.c</code> , <code>optimizer/path/joinrels.c</code> , <code>optimizer/plan/createplan.c</code>
Input	<code>Query *</code> struct
What happens	Generates all possible scan/join paths, costs each one using statistics from <code>pg_statistic</code> , picks the cheapest, converts it to a concrete plan tree
Output	<code>PlannedStmt *</code> — a full plan tree with nodes like <code>SeqScan</code> , <code>HashJoin</code> , <code>Sort</code>

Stage 6 — Executor (`src/backend/executor/`)

Files	<code>execMain.c</code> , <code>execProcnode.c</code> , <code>nodeSeqscan.c</code> , <code>nodeHashjoin.c</code> , <code>nodeSort.c</code> , <code>nodeIndexscan.c</code>
Input	<code>PlannedStmt *</code> plan tree
What happens	Walks the plan tree top-down. Each node type has its own file. Calls storage layer to fetch pages. Returns one tuple at a time (volcano/pull model)
Output	A stream of <code>TupleTableSlot *</code> — one row at a time

Stage 7 — Storage (`src/backend/storage/`)

Files	<code>buffer/bufmgr.c</code> → <code>access/heapam.c</code> → <code>smgr/smgr.c</code> → <code>smgr/md.c</code> → <code>file/fd.c</code>

Input	Page read/write requests from executor
What happens	<code>bufmgr.c</code> manages the shared buffer pool (8 KB pages in RAM). If page not in pool → calls <code>smgr</code> → <code>md.c</code> → <code>fd.c</code> to read from disk
Output	8 KB pages loaded into shared memory, tuples handed back to executor

Stage 8 — Result dispatch (`src/backend/tcop/` + `libpq/`)

Files	<code>dest.c</code> , <code>pqformat.c</code> , <code>pqcomm.c</code>
Input	Stream of <code>TupleTableSlot *</code> rows
What happens	Serialises each row into PostgreSQL wire protocol format (the "D" DataRow message), buffers and flushes over the socket
Output	Bytes on the wire → rows appear in your <code>psql</code> terminal

RLS (Row Level Security)

RLS is **not** a separate stage — it is injected much earlier but enforced inside the executor.

The journey has two parts:

Part 1 — Injection (Rewriter stage)

When the rewriter in `rewriteHandler.c` processes your query, it calls `check_enable_rls()` in `src/backend/utils/misc/rowsecurity.c`. If RLS is enabled on the table, it fetches the matching policies from `pg_policy` and injects them as extra `WHERE` qual conditions directly into your `Query` tree. You never wrote those conditions — Postgres silently added them.

Part 2 — Enforcement (Executor stage)

Those injected quals flow into the plan tree like any normal `WHERE` clause. Inside the executor, `ExecQual()` in `execExpr.c` evaluates them **per tuple** as rows come off the heap. A tuple that fails the RLS qual is silently dropped — the client never sees it. No error, no indication, just gone.

Key files:

```
src/backend/utils/misc/rowsecurity.c ← policy lookup + injection
src/backend/rewrite/rewriteHandler.c ← where injection is triggered
```

⇒ rewriteHandler.c

Query Struct:

```
typedef struct Query
{
    NodeTag      type;

    CmdType      commandType;    /* select|insert|update|delete|merge|utility */

    /* where did I come from? */
    QuerySource  querySource pg_node_attr(query_jumble_ignore);

    /*
     * query identifier (can be set by plugins); ignored for
     * equal, as it
     * might not be set; also not stored. This is the result
     * of the query
     * jumble, hence ignored.
     *
     * We store this as a signed value as this is the form i
     * t's displayed to
     * users in places such as EXPLAIN and pg_stat_statements.
     * Primarily this
     * is done due to lack of an SQL type to represent the full
     * range of
     * uint64.
     */
    int64        queryId pg_node_attr(equal_ignore, query_jumble_ignore, read_write_ignore, read_as(0));

    /* do I set the command result tag? */
    bool         canSetTag pg_node_attr(query_jumble_ignore);
```

```

Node      *utilityStmt;    /* non-null if commandType ==
CMD_UTILITY */

/*
 * rtable index of target relation for INSERT/UPDATE/DELETE/MERGE; 0 for
 * SELECT. This is ignored in the query jumble as unrelated to the
 * compilation of the query ID.
 */
int      resultRelation pg_node_attr(query_jumble_ignore);

/* has aggregates in tlist or havingQual */
bool     hasAggs pg_node_attr(query_jumble_ignore);
/* has window functions in tlist */
bool     hasWindowFuncs pg_node_attr(query_jumble_ignore);

/* has set-returning functions in tlist */
bool     hasTargetSRFs pg_node_attr(query_jumble_ignore);

/* has subquery SubLink */
bool     hasSubLinks pg_node_attr(query_jumble_ignore);

/* distinctClause is from DISTINCT ON */
bool     hasDistinctOn pg_node_attr(query_jumble_ignore);

/* WITH RECURSIVE was specified */
bool     hasRecursive pg_node_attr(query_jumble_ignore);

/* has INSERT/UPDATE/DELETE/MERGE in WITH */
bool     hasModifyingCTE pg_node_attr(query_jumble_ignore);

/* FOR [KEY] UPDATE/SHARE was specified */
bool     hasForUpdate pg_node_attr(query_jumble_ignore);

```

```

    /* rewriter has applied some RLS policy */
    bool          hasRowSecurity pg_node_attr(query_jumble_ignore);

    /* parser has added an RTE_GROUP RTE */
    bool          hasGroupRTE pg_node_attr(query_jumble_ignore);

    /* is a RETURN statement */
    bool          isReturn pg_node_attr(query_jumble_ignore);

    List          *cteList;          /* WITH list (of CommonTableExpr's) */

    List          *rtable;           /* list of range table entries */

    /*
     * list of RTEPermissionInfo nodes for the rtable entries
     * having
     *   perminfoindex > 0
     */
    List          *rteperminfos pg_node_attr(query_jumble_ignore);

    FromExpr      *jointree;         /* table join tree (FROM and
    WHERE clauses);
                                     * also USING clause for MERGE */

    List          *mergeActionList;  /* list of actions for MERGE (only) */

    /*
     * rtable index of target relation for MERGE to pull data. Initially, this
     * is the same as resultRelation, but after query rewriting, if the target
     * relation is a trigger-updatable view, this is the index

```

```

x of the expanded
    * view subquery, whereas resultRelation is the index of
the target view.
    */
    int          mergeTargetRelation pg_node_attr(query_jumble
_ignore);

    /* join condition between source and target for MERGE */
    Node          *mergeJoinCondition;

    List          *targetList;      /* target list (of TargetEntr
y) */

    /* OVERRIDING clause */
    OverridingKind override pg_node_attr(query_jumble_ignor
e);

    OnConflictExpr *onConflict; /* ON CONFLICT DO [NOTHING |
UPDATE] */

    /*
    * The following three fields describe the contents of th
e RETURNING list
    * for INSERT/UPDATE/DELETE/MERGE. returningOldAlias and
returningNewAlias
    * are the alias names for OLD and NEW, which may be user
-supplied values,
    * the defaults "old" and "new", or NULL (if the default
"old"/"new" is
    * already in use as the alias for some other relation).
    */
    char          *returningOldAlias pg_node_attr(query_jumble_i
gnore);
    char          *returningNewAlias pg_node_attr(query_jumble_i
gnore);
    List          *returningList; /* return-values list (of Tar

```

```

getEntry) */

    List      *groupClause;    /* a list of SortGroupClaus
e's */
    bool      groupDistinct;  /* is the group by clause dis
tinct? */

    List      *groupingSets;   /* a list of GroupingSet's if
present */

    Node      *havingQual;     /* qualifications applied to
groups */

    List      *windowClause;   /* a list of WindowClause's
*/

    List      *distinctClause; /* a list of SortGroupClaus
e's */

    List      *sortClause;     /* a list of SortGroupClaus
e's */

    Node      *limitOffset;    /* # of result tuples to skip
(int8 expr) */
    Node      *limitCount;     /* # of result tuples to retu
rn (int8 expr) */
    LimitOption limitOption;    /* limit type */

    List      *rowMarks;       /* a list of RowMarkClause's
*/

    Node      *setOperations;   /* set-operation tree if this
is top level of
                                * a UNION/INTERSECT/EXCEPT q
uery */

```

```

    /*
     * A list of pg_constraint OIDs that the query depends on
    to be
     * semantically valid
    */
    List      *constraintDeps pg_node_attr(query_jumble_ignore);

    /* a list of WithCheckOption's (added during rewrite) */
    List      *withCheckOptions pg_node_attr(query_jumble_ignore);

    /*
     * The following two fields identify the portion of the source text string
     * containing this query. They are typically only populated in top-level
     * Queries, not in sub-queries. When not set, they might both be zero, or
     * both be -1 meaning "unknown".
    */
    /* start location, or -1 if unknown */
    ParseLoc   stmt_location;
    /* length in bytes; 0 means "rest of string" */
    ParseLoc   stmt_len pg_node_attr(query_jumble_ignore);
} Query;

```

In the Postgres.c they are calling this querywrite() function:

```

List *
pg_rewrite_query(Query *query)
{
    List      *querytree_list;

    if (Debug_print_parse)
        elog_node_display(LOG, "parse tree", query,

```

```

        Debug_pretty_print);

    if (log_parser_stats)
        ResetUsage();

    if (query->commandType == CMD_UTILITY)
    {
        /* don't rewrite utilities, just dump 'em into result
list */
        querytree_list = list_make1(query);
    }
    else
    {
        /* rewrite regular queries */
        querytree_list = QueryRewrite(query);
    }

    if (log_parser_stats)
        ShowUsage("REWRITER STATISTICS");

#ifdef DEBUG_NODE_TESTS_ENABLED

    /* Optional debugging check: pass querytree through copy0
bject() */
    if (Debug_copy_parse_plan_trees)
    {
        List      *new_list;

        new_list = copyObject(querytree_list);
        /* This checks both copyObject() and the equal() rout
ines... */
        if (!equal(new_list, querytree_list))
            elog(WARNING, "copyObject() failed to produce an
equal rewritten parse tree");
        else
            querytree_list = new_list;

```

```

    }

    /* Optional debugging check: pass querytree through outfuncs/readfuncs */
    if (Debug_write_read_parse_plan_trees)
    {
        List      *new_list = NIL;
        ListCell  *lc;

        foreach(lc, querytree_list)
        {
            Query      *curr_query = lfirst_node(Query, lc);
            char        *str = nodeToStringWithLocations(curr_query);

            Query      *new_query = stringToNodeWithLocations(str);

            /*
             * queryId is not saved in stored rules, but we must preserve it
             * here to avoid breaking pg_stat_statements.
             */
            new_query->queryId = curr_query->queryId;

            new_list = lappend(new_list, new_query);
            pfree(str);
        }

        /* This checks both outfuncs/readfuncs and the equal() routines... */
        if (!equal(new_list, querytree_list))
            elog(WARNING, "outfuncs/readfuncs failed to produce an equal rewritten parse tree");
        else
            querytree_list = new_list;
    }

```

```

#endif                                     /* DEBUG_NODE_TESTS_ENABLED
*/

    if (Debug_print_rewritten)
        elog_node_display(LOG, "rewritten parse tree", querytree_list,
                           Debug_pretty_print);

    return querytree_list;
}

```

Here they are storing results in `querytree_list` variable.

Complete Investigation:

Q1 $\Rightarrow$ Format of the input (which we are passing to re-writer after parsing and analyzing)

After the **Parser** creates a raw parse tree and the **Analyzer/Semantic Analysis** enriches it into a **Query tree (Abstract Syntax Tree with resolved OIDs)**, this Query tree is passed to the **Rewriter**.

What the Rewriter Receives

The Analyzer outputs a `Query` struct (a C struct in PostgreSQL internals). For `SELECT * FROM customers`, it looks conceptually like:

```

Query {
  commandType: CMD_SELECT
  rtable: [
    RangeTblEntry {
      relid: 16384          ← OID of 'customers' table (resolved by analyzer)
      relkind: 'r'         ← regular relation
      eref: Alias { colnames: [id, name, email, ...] } ← * expanded
    }
  ]
}

```

```

    }
  ]
  jointree: FromExpr {
    fromlist: [RangeTblRef { rtindex: 1 }]
    quals: NULL          ← no WHERE clause
  }
  targetList: [
    TargetEntry { resno:1, expr: Var{id},    resname:"id"
  }
    TargetEntry { resno:2, expr: Var{name},  resname:"name"
  }
    TargetEntry { resno:3, expr: Var{email}, resname:"email"
  }
    ...          ← * is already expanded to actual
columns
  ]
  hasAggs: false
  sortClause: []
  limitCount: NULL
}

```

Every Function call is happening in `postgres.c` file where in each if condition postgres is calling functions for each phase starting from parsing.

Line No 50: `rewriteHandler.c`

```

/* We use a list of these to detect recursion in RewriteQuery
*/
typedef struct rewrite_event
{
    Oid          relation;          /* OID of relation having rul
es */
    CmdType      event;             /* type of rule being fired
*/
} rewrite_event;

```

When `RewriteQuery()` rewrites a query, the rewritten query might **trigger another rule**, which rewrites again — potentially **infinitely**.

So before firing a rule it checks:

*"Have I already fired a rule for this **same relation + same command type** combination in this chain?"*

It maintains a **list of** `rewrite_event` structs as a stack/history:

```
[ {customers, SELECT}, {orders, SELECT}, ... ]
```

Before firing a new rule → **scan the list** → if `{relation, event}` already exists → **recursion detected** → **error out**. If not → **push** `{relation, event}` onto list → rewrite → **pop** when done. That's it. Just a **"have I been here before?"** checklist.

```
static Query* fireRIRrules(Query *parsetree, List *activeRIRs)
```

! Ma'am asked about this:

```
void get_row_security_policies(Query *root, RangeTblEntry *rte, int rt_index, List **securityQuals, List **withCheckOptions, bool *hasRowSecurity, bool *hasSubLinks)
```

Line No 2294:

```
if (list_member_oid(activeRIRs, RelationGetRelid(re)))
    ereport(ERROR,
            (errcode(ERRCODE_INVALID_OBJECT_DEFINITION),
             errmsg("infinite recursion detected in policy for relation \"%s\"",
                    RelationGetRelationName(re))));
```

it is in the for each loop.

`activeRIRs` stores a **list of OIDs** (`List*` of `Oid`).

```
List *activeRIRs = NIL;
activeRIRs = lappend_oid(activeRIRs, RelationGetRelid(rel));
```

What: OID of the relation whose RLS policy is currently being applied.

Format: PostgreSQL `List*` — internally a linked list of `Oid` (which is just `unsigned int`).

```
activeRIRs → [ 16384, 16392, 16400 ]
               ↑       ↑       ↑
            customers orders audit ← their OIDs
```

	<code>rewrite_event</code> list	<code>activeRIRs</code> list
Where	<code>RewriteQuery()</code>	<code>ApplyRetrieveRule()</code>
Detects	Recursion in rule firing	Recursion in RLS policy / view expansion
Stores	<code>{relation OID + CmdType}</code>	<code>relation OID</code> only
Catches	e.g. a rule on <code>customers</code> rewrites to query <code>customers</code> again	e.g. a view/policy on <code>customers</code> expands back into <code>customers</code> again

`rewrite_event` guards the **rule rewriting loop**, `activeRIRs` guards the **RLS/view expansion loop** — two different recursion traps at two different steps.

My Complete Understanding of PostgreSQL Rewriter and RLS

When we run a query in PostgreSQL, it first hits `postmaster.c` which spins up a new backend process and hands control to `postgres.c`. This file is the main loop that drives all stages of query processing — from parser to executor. After the parser converts raw SQL text into a parse tree and the analyzer enriches it into a fully resolved `Query` struct (with OIDs, column references, and type information), this `Query*` pointer is passed to the rewriter.

How the Rewriter Detects Recursion

Inside `rewriteHandler.c` there is a function `static Query* fireRIRules(Query *parsetree, List *activeRIRs)`. This function receives the parse tree and a list called `activeRIRs`. As it traverses the query tree and encounters relations that have rules or RLS

policies attached, it adds the OID of each relation to the `activeRIRs` list before processing it. Before adding any OID it checks whether that OID already exists in the list. If it does, that means the same relation is being processed again in the same chain — which is a cycle — and PostgreSQL immediately throws an error. This is the mechanism used to detect infinite recursion during rewriting.

How RLS Flows Through PostgreSQL and Where It Gets Injected

The entry point for all rewriting is `QueryRewrite()` inside `rewriteHandler.c`. This calls `RewriteQuery()` which for SELECT queries calls `ApplyRetrieveRule()`. Inside `ApplyRetrieveRule()` the RLS logic is triggered by calling into `rowsecurity.c`.

Inside `rowsecurity.c` the main function is `get_row_security_policies()`. It first calls `RelationGetRowLevelSecurity()` to check whether RLS is actually enabled on the relation. Then it calls `get_policies_for_relation()` which fetches the applicable policies from the `pg_policy` catalog and filters them by command type — SELECT, INSERT, UPDATE, or DELETE. Once the policies are collected, `add_security_qual()` injects them as WHERE clause conditions into the query using the USING clause of the policy. For INSERT and UPDATE, `add_with_check_options()` additionally injects WITH CHECK quals. The helper `make_and_explicit()` is used to combine multiple policy conditions cleanly.

Back in `rewriteHandler.c`, `ApplyRetrieveRule()` takes the quals returned by `get_row_security_policies()` and merges them directly into the query's jointree, which is where the WHERE clause lives. This is how the RLS policy physically becomes part of the query.

The Exact Order and Why It Matters

The order of operations inside `ApplyRetrieveRule()` is: first call `get_row_security_policies()` to inject the RLS quals, and then call `fireIRRules()` to process the expanded query tree. The recursion check via `activeRIRs` happens inside `fireIRRules()` after the injection. This order is intentional — because the RLS policy itself can reference another relation or view, and that relation might loop back. If the cycle check happened before RLS injection, it would miss any recursion introduced by the policy itself.

How RLS Bypasses Are Handled

Inside the RLS code itself, before fetching any policies, PostgreSQL checks whether the current user is a superuser, whether the relation has RLS disabled, or whether the function is running as SECURITY DEFINER. If any of these conditions are true, `get_row_security_policies()` returns nothing — no policy, no quals injected, no expansion, and therefore no possibility of a cycle. Only when an actual policy exists does the rewriter proceed to expand and check for recursion via `fireIRules()`.